System Design Interview Vol. 2 (Alex Yu & Sahn Lam) — Problem Summaries

A study reference covering the 13 design problems from *System Design Interview – An Insider's Guide, Volume 2*. Each entry is broken into the problem itself, the high-level architecture, the deep-dive details an interviewer would probe, and the core algorithms/data structures involved.

> **Sourcing note:** The Volume 2 book was not in the uploaded files, so this summary is written from my knowledge of its chapters rather than extracted from the text. Overall designs and trade-offs are reliable, but the back-of-envelope numbers are approximate. If you upload the Volume 2 epub, I can verify the figures against the book.

---

1. Proximity Service

High-Level Problem
Build a Yelp/Google-Places-style service that returns businesses near a user's location within a chosen radius (roughly 0.5–20 km). Scale is ~100M daily users and ~200M businesses, so around 5K search QPS. The workload is read-heavy and latency-sensitive, while business updates can appear with a delay (next day is acceptable).

High-Level Design
Two services behind a load balancer. The **Location-Based Service (LBS)** is stateless and read-heavy, and handles `GET /search/nearby?lat&long&radius`. The **Business Service** handles business CRUD and detail views, which are write-light. Business data lives in a primary DB with read replicas, alongside a geospatial index that maps location cells to business IDs. LBS finds candidate IDs via the index, fetches details, and ranks by distance.

Design Deep Dive
The key decision is the geospatial index. A naive `WHERE lat BETWEEN ... AND long BETWEEN ...` query intersects two huge ranges and is too slow. Indexes fall into two families: hash-based (even grid, geohash) and tree-based (quadtree, Google S2, R-tree). **Geohash** recursively halves the world's lat/long ranges and base-32 encodes the result into a string, so nearby places share a prefix. Longer prefix means smaller cell; the radius picks the precision (~0.5 km → 6 characters, ~20 km → 4). Two edge cases need handling: neighbors across a cell boundary can have totally different prefixes, so LBS always queries the target cell plus its 8 neighbors, and if too few results come back, it drops the last character to widen the search. A **quadtree** is an in-memory tree that keeps subdividing a cell until it holds ≤100 businesses; it adapts to density (dense cities get small cells) and costs only a couple of GB, but it is rebuilt on server startup and is harder to update incrementally. Geohash is simpler to build and

update; quadtrees make "k nearest" easier. In production, the geohash → business-ID list is cached in Redis, the index table is replicated rather than sharded (it's small), and deployments span regions.

Algorithms & Data Structures
Geohash (recursive binary partitioning + base-32 encoding), quadtree (adaptive spatial subdivision), Google S2 (Hilbert-curve cell covering), and the Haversine formula for final distance ranking. Redis key-value caches back both the index lookup and business details.

---

2. Nearby Friends

High-Level Problem
Show a user which of their friends are currently within a given distance (e.g., 5 miles) and keep that list live as people move. Location refreshes roughly every 30 seconds. With ~100M DAU and ~10M concurrent users, that is on the order of 330K location updates/sec — a very write-heavy, real-time fan-out problem.

High-Level Design
Clients hold a **WebSocket** connection to stateful WebSocket servers. Each location update is (1) saved to a **location cache** (Redis, with a TTL so inactive users vanish), (2) appended to a **location history DB** (Cassandra-style, write-optimized), and (3) published to a **Redis Pub/Sub channel unique to that user**. Every online friend's WebSocket handler is subscribed to that channel, so it receives the update, computes distance, and pushes it to its client only if the friend is within the radius.

Design Deep Dive
WebSocket servers are stateful, so removing one means draining connections gradually. The Redis Pub/Sub layer is the hard part: it holds millions of channels and dominates CPU (on the order of 100+ servers for ~14M pushes/sec). It is sharded by **consistent hashing**, with the hash ring stored in a service-discovery system (etcd/ZooKeeper) so WebSocket servers know which Pub/Sub node owns each user's channel. Because resizing this cluster forces mass resubscription, it is over-provisioned and resized only during low-traffic windows. Subscriptions are limited to friends who are online and have the feature enabled. When someone logs in, the server loads friends' last-known locations from the cache for the initial view. A "nearby strangers" extension uses geohash-based channels (users publish to their cell's channel and subscribe to neighbors).

Algorithms & Data Structures
Pub/sub channels per user, consistent hashing for Pub/Sub sharding, TTL'd key-value location cache, geohash (for nearby-stranger channels), and Haversine for distance filtering.

---

3. Google Maps

High-Level Problem
Design a navigation service covering three capabilities: **location tracking**, **route navigation** (with ETA

and rerouting), and **map rendering**. It must serve ~1B users on mobile, give accurate directions and ETAs, and keep bandwidth and battery use reasonable.

High-Level Design

Three subsystems. **Map rendering:** the world is pre-split into tiles at ~21 zoom levels (each level has 4× the tiles of the previous), tiles are stored in object storage and served through a CDN; the client requests the tiles for its viewport and zoom, so rendering is just static file serving. **Navigation:** a route planner takes origin/destination → geocoding → shortest-path service → ETA service → ranker (filter tolls/highways, etc.). **Location service:** clients batch their location (about every 15 seconds) to the backend, which streams it via Kafka into a write-heavy location store, feeding traffic estimation and personalization.

Design Deep Dive

Road data is turned into a **graph** and cut into **routing tiles**, hierarchically (local, arterial, highway levels), stored in object storage and loaded lazily, so a long route uses coarse highway tiles in the middle and fine tiles near endpoints instead of loading the whole planet. Routing uses **Dijkstra/A*** on this tiled graph. The ETA service combines the route's distance with live and historical traffic (ML-driven). For **adaptive rerouting**, the server tracks which routing tiles each active navigating user is on; when traffic changes on a tile, it finds affected users and pushes updates over **WebSocket**. Map clients prefer **vector tiles** (shapes + styles rendered client-side with WebGL) over raster images: smaller payloads and smooth zooming.

Algorithms & Data Structures

Web-Mercator map projection, geohash/tile addressing, graph + adjacency lists for the road network, Dijkstra and A* for shortest path (with hierarchical tile-based search), Kafka for location ingestion, and precomputed tile pyramids behind a CDN.

---

4. Distributed Message Queue

High-Level Problem

Design a Kafka-like durable message queue: producers publish, consumers read, the same message can be consumed multiple times, ordering holds within a partition, data is retained for a configurable period (e.g., two weeks), and delivery semantics (at-most-once, at-least-once, exactly-once) are configurable. It must offer high throughput with tunable latency.

High-Level Design

Core concepts: **topic → partitions → messages**, each message with an **offset**. **Brokers** host partitions; **producers** route messages to a partition (by key hash or round-robin); **consumers** in a **consumer group** divide a topic's partitions among themselves so each partition is read by one consumer in the group. A **coordination service** (ZooKeeper/etcd) holds metadata, tracks broker liveness, and elects partition leaders; a separate state store tracks consumer offsets.

Design Deep Dive

Storage is a **write-ahead log**: each partition is a sequence of append-only **segment files**, with one

active segment taking writes. Disk is fine because access is sequential (and leans on the OS page cache and zero-copy transfer). Batching at the producer and broker amortizes I/O, trading latency for throughput. **Replication:** each partition has a leader and followers; the **in-sync replica (ISR)** set contains followers that are caught up, and a write is committed once all ISRs have it. Producer `acks` (0 / 1 / all) lets users trade durability for latency. Consumers **pull** rather than being pushed to, which lets them control rate and batch size. A **group coordinator** handles joining, heartbeats, and **rebalancing** when members come or go. Scaling means adding brokers and partitions, with a replica distribution plan to avoid hotspots.

Algorithms & Data Structures
Append-only write-ahead log with segmentation, offset-based indexing, ISR-based replication, hash partitioning by key, and a coordinator-based group rebalance protocol.

---

5. Metrics Monitoring & Alerting System

High-Level Problem
Build an internal system that collects metrics (CPU, memory, request counts, etc.) from huge fleets of servers, stores them, visualizes them, and fires alerts. Scale: ~10M active metrics (e.g., 1,000 pools × 100 machines × 100 metrics), with retention tiering (raw for ~7 days, 1-minute resolution for ~30 days, 1-hour resolution for a year).

High-Level Design
Five stages: data collection → transmission → storage → alerting → visualization. Metrics are modeled as **time series** (name, labels/tags, timestamped values) and stored in a **time-series database** (InfluxDB/Prometheus-style) since the workload is constantly write-heavy with spiky reads. A query service reads the TSDB for dashboards (Grafana-like) and for the alert manager, which evaluates rules and sends notifications.

Design Deep Dive
Collection can be **pull** (collectors scrape endpoints discovered via etcd/ZooKeeper, Prometheus-style) or **push** (agents on each server send to collectors, CloudWatch/Graphite-style); the pull model eases health checks and debugging, while push suits short-lived jobs and firewalled networks. Collector clusters scale using **consistent hashing** to assign targets. **Kafka** sits between collectors and storage to decouple them and absorb bursts, and stream processors (Flink/Spark) can pre-aggregate before writing. Storage tricks include **delta encoding** for timestamps, **downsampling** old data to coarser resolution, and moving cold data to cheaper storage. Alerting: rules defined in config (YAML) are cached, the alert manager evaluates them against the query service, tracks alert state, de-duplicates and merges alerts, and pushes events through Kafka to email/PagerDuty/webhooks with retry.

Algorithms & Data Structures
Time-series data model with tag indexing, delta/compression encoding, downsampling/rollups, consistent hashing for collector assignment, and Kafka-buffered stream aggregation.

---

6. Ad Click Event Aggregation

High-Level Problem
Aggregate ad-click events to answer: "How many clicks did ad X get in the last M minutes?", "What are the top 100 most-clicked ads in the last M minutes?", with filters like country or IP. Scale is ~1B clicks/day (~10K QPS average, ~50K peak), with strong correctness requirements because the data drives billing.

High-Level Design
Click events flow into a **Kafka** queue; a streaming **aggregation service** (a MapReduce-style DAG of Map → Aggregate → Reduce nodes) counts clicks per ad per minute and writes results to a second Kafka topic, and a sink writes them to a database (Cassandra-style) queried by a **query service**. Raw events are also archived to cold storage so history can be replayed or recomputed.

Design Deep Dive
The design follows a **Kappa architecture**, using one streaming path instead of Lambda's separate batch and speed layers, with replay from raw data for fixes plus end-of-day **reconciliation** to catch discrepancies. Time handling matters: **event time** (when the click occurred) is preferred over processing time for accuracy, with **watermarks** granting late events a grace period; windows are tumbling (per-minute) and sliding (last M minutes). **Exactly-once** processing is achieved by committing the upstream Kafka offset and the downstream result atomically, e.g. by storing the offset with the output so a restarted aggregator doesn't double-count. Scaling: partition by `ad_id`, run aggregators as a consumer group, and give hot ads extra nodes via a resource manager (YARN/Kubernetes). Aggregator state is periodically **snapshotted** for fast recovery. A star-schema with filter IDs pre-aggregates common filter combinations.

Algorithms & Data Structures
MapReduce DAG (map → aggregate → reduce), min-heap of size N for top-N merging, tumbling/sliding windows, watermarks for late data, Kappa architecture, and offset + state atomic commits for exactly-once.

---

7. Hotel Reservation System

High-Level Problem
Design an Airbnb/Booking-style system covering hotel/room browsing, searching availability, and making reservations. Scale is modest (~5K hotels, ~1M rooms, roughly 240K reservations/day ≈ 3 TPS) but **correctness is critical**: no double-booking, and a safe checkout flow.

High-Level Design
Microservices: hotel service (static hotel/room info, cache-friendly), rate service (prices by date), reservation service (create/cancel/status), inventory service (availability), and a payment service. Because the workload needs ACID transactions and relational queries, a **relational DB** is used.

Reservations target a **room type** (not a specific room), and an inventory table tracks `total_inventory` and `total_reserved` per (hotel, room type, date).

Design Deep Dive

Two concurrency problems dominate. **Same user double-clicking** is solved with **idempotency**: a `reservation_id` generated upfront is a unique key, so a retry hits a unique-constraint violation instead of creating a second booking. **Two users racing for the last room** is solved with locking. Pessimistic (`SELECT ... FOR UPDATE`) is simple but contention-heavy, optimistic (version number check-and-retry) works well at low contention, and a DB **check constraint** (`total_inventory - total_reserved ≥ 0`) is a simple safety net. Hotels often allow ~10% **overbooking**, built into the inventory check. The reservation lifecycle is a state machine (pending → paid → refunded/canceled/rejected). To scale, data is **sharded by** `hotel_id`, and a Redis inventory cache serves read-heavy availability checks with the DB as the source of truth (kept consistent via change-data-capture). Splitting the reservation and inventory services creates a distributed-transaction problem, handled by **2PC or Saga**, or avoided by keeping them in one transactional store.

Algorithms & Data Structures

Idempotency keys with unique constraints, optimistic and pessimistic locking, DB constraints, hash sharding by `hotel_id`, reservation state machine, and Saga/2PC for cross-service consistency.

---

8. Distributed Email Service

High-Level Problem

Design a Gmail-scale email system for ~1B users: send, receive, store, organize into folders/labels, search, and handle attachments. It must sustain on the order of 100K sends/sec and petabytes of new storage per year, with high reliability, anti-spam, and global availability.

High-Level Design

Sending uses **SMTP**: the web server hands the email to an outgoing queue, a worker does spam/virus checks and relays via SMTP to the recipient's mail server (found via a **DNS MX lookup**). Receiving mirrors this: inbound SMTP servers validate, queue, and process mail, writing to storage and notifying online clients through **WebSocket** real-time servers. Storage is split: **metadata DB** (headers, folders, flags), **attachment object store** (S3-like), a **search store**, and a distributed cache for hot mailbox data.

Design Deep Dive

No off-the-shelf DB is a perfect fit (emails are variable-size, write-heavy, and need strong consistency), so the design is a **wide-column store** (Bigtable-style) **partitioned by** `user_id`, with `email_id` (time-ordered ID) as clustering key so a user's mailbox is contiguous and sorted. Queries needed by the UI (by folder, read/unread, label) get denormalized tables, plus counters for unread counts. Large attachments go to object storage, not the DB. **Search** is write-heavy (every email is indexed on arrival) with comparatively few reads, so indexing is decoupled via Kafka into per-user **inverted indexes**, either Elasticsearch or a custom LSM-tree-based engine. **Deliverability** requires dedicated and "warmed-up" IPs, separating transactional from marketing traffic, handling bounces and feedback loops, and email

authentication via **SPF, DKIM, and DMARC**. Availability comes from multi-datacenter replication with failover.

Algorithms & Data Structures
SMTP + DNS MX routing, user-partitioned wide-column storage with time-ordered clustering keys, inverted index with LSM-tree storage for search, Kafka for async indexing, and SPF/DKIM/DMARC for sender authentication.

---

9. S3-like Object Storage

High-Level Problem
Build an Amazon S3-style service supporting buckets, immutable objects, versioning, upload/download, listing by prefix, and large-file multipart upload. Targets: ~100 PB of data, very high durability (six nines or better), ~four nines availability, and storage efficiency.

High-Level Design
Separate **metadata** from **data**. API servers (behind a load balancer, with IAM for auth) front two stores: a **metadata store** (bucket and object tables mapping object names to object IDs/versions) and a **data store** made of a **data routing service**, a **placement service** (keeps a virtual cluster map of data nodes and picks where replicas go), and **data nodes** (store bytes on disk and heartbeat to the placement service). An upload gets a UUID, is routed to a primary data node, and is replicated before success is returned.

Design Deep Dive
Storing each object as its own file wastes inodes and blocks, so data nodes **append many small objects into large log-style files** that become read-only once full; a per-node mapping table (SQLite) records `object_id → (file, offset, size)`. **Durability:** 3× replication is simple but adds 200% overhead; **erasure coding** (e.g., 8 data + 4 parity chunks) cuts overhead to ~50% and survives multiple losses, at the cost of extra compute and slower reads/repairs. Checksums at the object and chunk levels detect corruption. The metadata DB is sharded by `(bucket, object name)`; **listing** by prefix is tricky across shards and is often handled with a separate index. **Versioning** stores each version under a time-ordered version ID, with delete markers instead of physical deletes. **Multipart upload** lets clients upload parts in parallel and assemble them on completion. Deletion is lazy, and background **garbage collection/compaction** copies live objects into fresh files to reclaim space.

Algorithms & Data Structures
Reed-Solomon erasure coding, checksums (e.g., MD5), a virtual cluster map with failure-domain-aware placement, log-structured append-only data files with offset mapping, sharded metadata, and compaction-based garbage collection.

---

10. Real-time Gaming Leaderboard

High-Level Problem

Show the top players, a player's own rank, and players ranked around them, updated in real time as scores change, for a game with ~5M DAU (and a stretch scenario of hundreds of millions). Leaderboards may reset monthly. Writes are bursty and reads must be instant.

High-Level Design

A relational table with `ORDER BY score DESC` fails because finding a rank means scanning or sorting the table. Instead, the game server (authoritative, to prevent cheating — never trust clients to submit scores) updates a **Redis Sorted Set** (`ZADD` / `ZINCRBY`), and the leaderboard service reads from it with `ZREVRANGE` (top N), `ZREVRANK` (a user's rank), and a rank-window query for "players around me". MySQL stores user profiles and score history as the durable source of truth, so Redis can be rebuilt. Monthly boards use one key per period.

Design Deep Dive

Redis sorted sets are backed by a **skip list** plus a hash table, giving O(log N) updates, rank lookups, and range reads, which comfortably handles 5M-user scale on one node with replicas for availability. For a much larger scale, sharding is needed. **Fixed (score-range) partitioning** puts, say, scores 1–100 in one shard, 101–200 in the next: the top-10 comes from the top shard, and a user's exact rank equals the counts of all higher shards plus their rank within their own shard. **Hash partitioning** (Redis Cluster) balances load evenly but forces a **scatter-gather** across all shards for top-K and makes exact rank expensive, so at huge scale the product often shows an approximate **percentile** instead. DynamoDB-style alternatives use partition-key sharding with a secondary index. Kafka can fan score events out to analytics and notifications. Old months get archived and cleared from Redis.

Algorithms & Data Structures

Redis sorted set (skip list + hash table), score-range vs. hash sharding, scatter-gather top-K merging with a heap, and percentile-based approximate ranking at extreme scale.

---

11. Payment System

High-Level Problem

Design the backend for an e-commerce payment flow: **pay-in** (customer pays the platform) and **pay-out** (platform pays sellers). Volume is modest (~1M transactions/day ≈ 10 TPS), but the requirements are stringent: no lost or double payments, strong consistency, security/PCI compliance, and tolerance for flaky external providers.

High-Level Design

A **payment service** accepts payment events, writes a payment order, and hands it to a **payment executor**, which calls an external **PSP** (Stripe, PayPal, etc.). To stay out of PCI scope, card data is collected on the PSP's **hosted payment page**, so the platform only sees a token; the PSP reports the result via **webhook**. Results update the **ledger** (double-entry bookkeeping) and **wallet** (seller balances). Core tables: `payment_event` , `payment_order` (with a status state machine), and ledger entries.

Design Deep Dive
Payments are the classic case where "exactly once" is built as **at-least-once retries + idempotency**. Retries use exponential backoff with jitter and hit a retry queue, with persistent failures routed to a **dead-letter queue** for investigation. The idempotency key (derived from the payment order ID) is sent to the PSP and enforced via DB unique constraints, so a retry never double-charges. Within the system, strong consistency needs synchronous replication, while cross-service updates use **Saga/TC-C** patterns or a single transactional boundary. Since the PSP is an external system, nightly **reconciliation** compares the PSP's settlement file against internal records; mismatches are classified (auto-fixable, manually fixable, unclassifiable) and resolved. Security layers include TLS, encryption at rest, tokenization, fraud detection, and rate limiting.

Algorithms & Data Structures
Idempotency keys, exponential backoff with jitter, double-entry ledger, payment state machine, dead-letter queues, reconciliation jobs, and Saga/two-phase protocols for distributed consistency.

---

12. Digital Wallet

High-Level Problem
Design a PayPal/Venmo-style wallet supporting balance transfers between users at very high scale (~1M TPS target), with ACID guarantees (no lost or duplicated money), reliability around 99.99%, and full auditability/reproducibility of every balance.

High-Level Design
Start with an in-memory key-value store (Redis cluster) holding balances, partitioned by user account with **consistent hashing**; a **transfer** is a debit on one partition and a credit on another. A coordinator service accepts transfer requests and drives the cross-partition operation. At ~1,000 TPS per node, ~1M TPS implies ~1,000 nodes, which makes cross-partition atomicity the central problem.

Design Deep Dive
Three approaches to distributed transactions are compared. **Two-phase commit (2PC)** is strongly consistent, but the coordinator is a bottleneck/single point of failure and holds locks during the protocol. **Try-Confirm/Cancel (TC/C)** works at the application level: *Try* reserves (debits A), then *Confirm* completes the credit or *Cancel* undoes the debit, with no long-held locks. **Saga** chains local transactions with compensating actions, giving eventual consistency. The best answer is **event sourcing**: instead of storing balances, store an immutable, ordered log of **commands → events**; a deterministic state machine validates a command, emits events, and applies them to update state. This gives a built-in audit trail and the ability to **replay** history to rebuild or verify any past state, with **CQRS** separating the write path from read-only views. For performance, each shard keeps its command/event log in **append-only local files** (memory-mapped), keeps state in a local embedded DB, takes periodic **snapshots** for fast recovery, and replicates the log across nodes with **Raft** so a failed leader is replaced with an identical history. Cross-shard transfers use Saga/TC-C on top.

Algorithms & Data Structures
Consistent hashing for account partitioning, 2PC, TC/C, Saga, event sourcing with CQRS, a deterministic state machine, snapshots, memory-mapped append-only logs, and Raft for replication and leader election.

---

13. Stock Exchange

High-Level Problem
Design an exchange that accepts buy/sell **limit orders**, matches them, and publishes market data. Scale is tens of thousands of orders/sec (~1B/day, bursting well above average), across ~100 symbols. The defining requirements are **ultra-low, predictable latency**, strict fairness/determinism, and 99.99%+ availability, since errors and downtime are extremely expensive.

High-Level Design
Order flow: **client gateway** (auth, rate limiting, FIX protocol) → **order manager** (risk and wallet checks) → **sequencer** (stamps each event with a global sequence ID and timestamp) → **matching engine** → executions back through the sequencer to the order manager and to the **market data publisher** (order book levels, candlestick charts) and a **reporter** for compliance records. The "hot path" (gateway → matching → response) is kept minimal; other work happens off the critical path.

Design Deep Dive
The **order book** keeps buy and sell sides as price levels, each level a **doubly linked list of orders in arrival order** (FIFO). A hash map from order ID to order node makes cancels O(1), and matching just walks from the best price's list head, giving **price-time priority**. For latency, the design avoids network hops by colocating components on one server where possible, communicating via **shared memory (mmap) event stores** instead of Kafka, and running the matching engine **single-threaded and CPU-pinned**: no locks, no context switches, and fully **deterministic**, so the same sequenced input stream always produces the same outputs. Determinism plus **event sourcing** (the sequenced event log is the source of truth) enables **hot-warm replication**: a standby engine replays the same log and takes over on failure. Market data is distributed to subscribers via fast fan-out (e.g., multicast), and risk checks sit before the sequencer so invalid orders never enter the engine.

Algorithms & Data Structures
Order book (price-level map + doubly linked list per level + order-ID hash map), FIFO price-time matching, the sequencer for total ordering, event sourcing with deterministic replay, mmap-based ring buffers/event bus, single-threaded matching with CPU pinning, and primary/standby (hot-warm) failover.

Reference: System Design Interview – An Insider's Guide, Volume 2, by Alex Yu and Sahn Lam (13 chapters). Written from knowledge of the book; the Volume 2 text itself was not available in this session, so verify exact figures against the original.